

Seed, Generate, Reduce, Repair, Gate: A Closed CIRCT Bug Loop

Abhinav Venkata Kota, Adithya Jillellamudi, Priyesh Shukla

International Institute of Information Technology, Hyderabad

{abhinav.kota, adithya.jillellamudi}@students.iiit.ac.in, priyesh.shukla@iiit.ac.in

Abstract—CIRCT is an open-source compiler that turns hardware descriptions into Verilog, so a defect in it can corrupt a chip unnoticed. Agents that repair CIRCT feed on its issue tracker, which only scarce maintainer attention drains. We built a loop that makes its own supply: an agent reads one of 187 CIRCT fix commits mined from 24 months, 171 rebuildable from a prebuilt SDK, names the class of mistake it corrected, and writes inputs probing for it elsewhere. Fixed tools, not the model, build, judge, minimise, deduplicate and repair; a named person approves every filing. CHIA’s CIRCT image compiles out the compiler’s own consistency checks, so nothing can fire; restored, they reach 536 of 555 compiler objects and leave CHIA’s test gate at 1,119 passes and 0 failures, identical with them off. Replaying six real CIRCT failures exposed two defects that would have misrouted three of four bugs. We release the loop as CHIA nodes and tools; the pre-registered campaign against a mutation fuzzer is running, and will report its maintainer-confirmed bug count.

I. INTRODUCTION

CIRCT is an open-source compiler for hardware. A designer writes how a circuit should behave, in FIRRTL (the form the Chisel hardware language produces) or in SystemVerilog, and CIRCT lowers it to Verilog, which synthesis and layout tools turn into a chip. It is built on MLIR, the LLVM project’s compiler framework, and inherits LLVM’s way of testing: a regression test carries the command to run and the output to expect, run by a tool called `lit`. A CIRCT bug that crashes the compiler is an inconvenience, seen and worked around; a bug that makes it emit wrong Verilog is not: the output is well formed, passes the tools below it, and can reach fabrication unnoticed.

CHIA [1] is the agent framework this hackathon is built on. Its issue solver for CIRCT chains six phases, from assessing an issue to reproducing, fixing, verifying against the whole test suite, repairing regressions and writing the pull request; on 16 CIRCT issues it fixed five, three of them merged.

Its supply is the constraint. Over the past year CIRCT received 82 bug-labelled issues and closed 65, so the project is neither short of reports nor ignoring them; what does not drain is the backlog of 101 open ones, at a median age of 911 days. The scarce resource is maintainer attention, and an agent that empties the queue faster creates none. What would help is a bug that arrives already minimised, already checked against what is known, already carrying a candidate fix, from a source that files nothing else. Past fixes are the unspent information: a commit that corrects one wrong assumption inside a pass usually leaves the same assumption at other sites, and CIRCT ships no fuzzing infrastructure to find them.

This paper describes a loop that manufactures its own supply and then withholds most of it: the loop as built (Section II), the preconditions we measured and repaired first (Section III), a pre-registered head-to-head against a mutation fuzzer (Section IV), and its result (Section V).

II. THE LOOP

Fig. 1 names the stages and marks the ones the agent runs; tools decide everything else, so no number in the result comes from inside the model. One model serves every agent stage, Gemini 3.8 Flash through CHIA’s Vertex backend, and no stage can build a client for it unless one environment variable is set for that run, so a test or a dry run cannot reach the model.

Read the seed (1). A seed is one of 187 CIRCT commits from the last two years that look like bug fixes, with its diff and the test its author added or changed. The agent reads both, states in one sentence the root-cause class the fix corrected, and names other sites where the same assumption still stands. The record carries that diff and those tests, so the worker needs no clone of CIRCT’s history.

Write probing inputs (2). The seed’s own test decides the language, because its command line names the tool through which the bug was reachable: `circt-opt`, which runs a named sequence of passes over an intermediate file; `firtool`, the driver from FIRRTL to Verilog; or the SystemVerilog front end. About 127 of the 187 seeds sit in code no FIRRTL input can reach, so a FIRRTL-only generator would miss most of the corpus. The agent reads the compiler’s source and writes each candidate through tools of its own, and runs no command.

Build (3). CIRCT compiles against one pinned commit of LLVM and MLIR, and Section III explains how the loop avoids building those from source. The build compiles in CIRCT’s assertions, the internal checks its own authors wrote, which stop the compiler the moment an assumption breaks.

Judge (4). An oracle is the rule that decides whether an input found a bug. The primary one admits no argument: the compiler crashed, or an assertion fired. A second runs the same design through CIRCT’s simulator and through Verilator, and treats disagreement as suspicious; it reports only, because the two disagree routinely over uninitialised and reset signals.

Reduce (5). A crashing input is usually large and mostly irrelevant to the crash. `circt-reduce`, which CIRCT ships, deletes parts of it and keeps each deletion while the failure survives; a textual reducer takes over where that tool dies on the input itself.

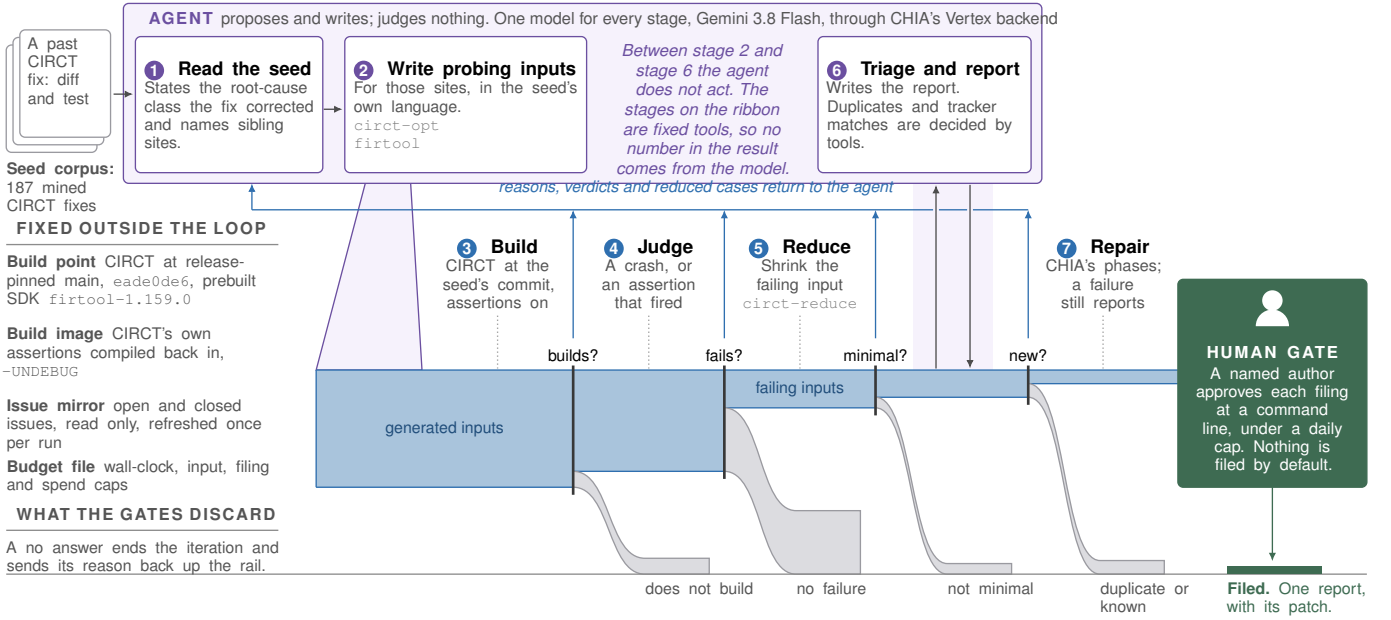


Fig. 1. Seeded CIRCT bug loop. The agent holds the violet band and acts three times only, at stages 1, 2 and 6; the blue ribbon below it is fixed apparatus, so no number in the result comes from the model. The ribbon is the candidates in flight, pinched at each gate by a question with two answers: the share answering no peels onto the floor, and its reason climbs the rail back to the agent. Widths are illustrative, not measured.

Triage and report (6). Many failures are one failure, and deciding which are the same is mechanical: a fingerprint is the assertion text with its source location, or, for a crash, the symbolised frames left once the crash handler's prologue is stripped. Survivors are matched against a local mirror of CIRCT's open and closed issues and against commits made since the build point. The agent's share is the last part, writing the report, and no gate reads what it concluded.

Repair (7). CHIA's phase chain runs unaltered but for one added branch selecting the backend the rest of the loop uses, and an entry point taking a local report in place of a GitHub issue. Repair is restricted to crashes and assertion failures, where nobody has to argue about what the right answer was.

The gates. Between stages sit questions with two answers, the bars that pinch the ribbon in Fig. 1. A no ends the iteration and returns its reason to the agent with the verdicts and reduced cases. The last is a gate of four questions, each answered by a tool: does the failure reproduce in a fresh process on another worker, is the case minimal, is the input valid, and is it new. The third is three parse-and-verify commands, one per input language, and a clean rejection makes the input the loop's own fault rather than the compiler's. Then the gate is a person: a named author approves every filing at a command line, under a cap on filings per day, and nothing is filed by default.

III. MEASURED PRECONDITIONS

The loop has to build CIRCT at arbitrary points of its history, and the compiler has to be able to complain. Neither held when we started. Table I is what we measured on the image the loop now runs; this section gives the conclusions.

Building the past cheaply. CIRCT records the exact LLVM commit it expects, the pin; a build against any other fails outright rather than degrading, and compiling that LLVM from

TABLE I
WHAT WE MEASURED ON THE IMAGE THE LOOP RUNS

Measurement	Result
<i>Building the past cheaply</i>	
Fix-shaped commits mined from 24 months, the seed corpus	187
Of those, seeds with an exactly matching prebuilt SDK	171
Pin windows with no matching release, worst 16 days	16 of 63
<i>Letting the compiler complain</i>	
Compile commands carrying -UNDEBUG	778 of 778
Object files carrying the assertion handler again	536 of 555
CHIA's whole test gate, assertions on and then off	1,119 pass, 0 fail, identical
What the assertions cost: wall clock, bytes, per probe	18%, 25%, 4%
Build cost of the line tables that name the crash site	none measurable
Tests <code>lit</code> discovers, once its source root is the SDK	1,390, 0 errors

source costs hours per seed. It also publishes an SDK, a tarball of prebuilt LLVM and MLIR libraries recording its own pin, and where an SDK shares a seed's pin CIRCT rebuilds in minutes: of 187 fix-shaped commits mined from 24 months, 171 match exactly.

The same mechanism fixes where the loop runs: not the newest commit of CIRCT's main branch, whose pin has no published release today, but release-pinned main, the newest commit whose pin has one, which for this image is 8 commits and 4.8 days behind the head. A long lag mechanically lowers the filing rate, because more candidates come back already fixed.

Letting the compiler complain. Optimised builds delete assertions by defining the macro `NDEBUG`, and CHIA's published CIRCT image is such a build: its binaries hold no reference to the assertion handler, so a loop whose oracle is a failed assertion would find nothing under it, whatever the agent wrote. One compiler flag, `-UNDEBUG`, puts them back, and the image the loop runs is that build at release-pinned main, with the flag

in every compile command and the handler back in all but 19 of its 555 compiler objects, which assert nothing.

Whether the restored checks fire on CIRCT’s own tests is what the oracle rests on, so we built the image twice, differing in that flag alone, and ran CHIA’s whole test gate in both: the two runs pass the same 1,119 tests and fail none. The set of tests that pass with assertions compiled out and fail with them compiled in is therefore empty, so a probe that trips an assertion trips it because of what the probe does, and Table I prices the flag.

One defect had to be repaired first: at CIRCT commits after mid-2026 a test directory’s configuration file raises while `lit` is discovering tests, and CHIA’s exclusion of that directory is applied after discovery, so the gate discovered nothing. Pointing the build at the SDK as its MLIR source root, one flag, restores discovery. The binary still calls itself an optimised build, so every report names the flag.

Tested on real failures. The oracle and its fingerprints were calibrated on real output: six past CIRCT bugs, rebuilt at the commit before their fix and recorded with input, argument vector and error output, in three classes across two SDK releases at 14s to 601s of build each. Two defects in the fingerprinting rules appeared only there: the name normaliser erased every frame in an anonymous namespace, which is where CIRCT’s passes live, and the scope test read an inlined frame instead of the physical one, putting three of the four real bugs outside the repair stage’s reach. Both are fixed, and each fixture now pins the corrected answer.

IV. EVALUATION DESIGN

The experiment asks whether reading a past fix makes an agent better at finding new bugs than mutating the same tests at random. Two arms run the same harness, oracles, reducer and gate, and only the generator differs: the seeded arm is stages 1 and 2 above, and the mutation arm mutates the same seed tests syntactically without ever seeing the diff or the root-cause class, following Mut4All’s design for mutators synthesised by a language model [2].

Both arms spend the same pre-registered budget, whose unit is one elapsed wall-clock second of an arm’s fixed window: they run one after the other, each for four hours, on the same cluster at the same concurrency. Safety caps bound damage rather than effort, at 2,000 generated inputs per arm per day, 3 filings per day and 10 in the campaign, every probe held to 60 seconds and 4 GiB, and a hard cap of 200 US dollars on model spending stops both arms. All of it is committed before the campaign starts, so no threshold can be chosen afterwards.

The headline is the number of distinct bugs a CIRCT maintainer confirmed, by labelling, commenting on or fixing them, within that budget; our own oracle agreeing with itself produces a candidate, not a bug. Table II names the five secondary counts.

Deduplication. Two candidates are one bug when a single primary fingerprint is equal, computed after the crash handler’s own prologue is stripped from the trace. On a hand-labelled set of 22 candidate pairs the rule collides on 1 of the 11 distinct

pairs and falsely merges 1 of the 11 duplicate pairs; both rates are printed beside the headline, since a collision inflates that count and a false merge deflates it.

Known issues. A mirror of the tracker, refreshed once per run, answers whether a candidate is already reported. GitHub stops paginating an issue listing at its hundredth page, which the loop met as an error and not as truncation, so the mirror walks newest first, then oldest first, and unions the two: 600 issues cost 35 requests and half a minute.

Contamination. CIRCT’s history sits in the training data of any model we can use, so the agent may reproduce a fix it memorised. Every candidate is screened against fixes made after its seed, at the level of the symbols the seed’s fix touched: that screen flags 77% of the corpus where the same screen at file level flags 96%. A real crash is a real crash, so the flag qualifies a candidate and reports it apart rather than excluding it. The repair stage runs CHIA’s executor unchanged and is not screened. We post the method to the project’s forum before the campaign starts, and leave issues marked as good first issues to human newcomers.

Failure taxonomy. Every candidate reaching the gate is recorded with its verdict, one of unreproducible, not minimal, invalid input, duplicate, undecided or new bug, and every repair attempt with the phase that failed, so the record shows where budget went without return.

V. RESULTS

Table II is filled from the artefact the loop renders out of its own database, one group of rows per table there, and every cell is *pending* until the pre-registered window closes. A zero is reportable and would be reported: pre-registration makes such an outcome publishable, and it would say that CIRCT’s assertion surface is not reachable at this budget.

VI. RELATED WORK

Agentic repair. SWE-bench [3] scores agents on real GitHub issues and set the pattern this work inherits; CHIA’s issue solver [1] applies it to a hardware compiler. Both draw from a queue; we add the stage that fills one.

Compiler fuzzing. CSmith [4] generates random C programs and found bugs throughout production C compilers. For MLIR, MLIRod mutates along operation dependencies [5], Nüwa adds generation driven by a language model [6], DESIL targets silent wrong-output bugs [7], and FLEX interleaves learning with exploration, finding 80 previously unknown bugs in a 30-day campaign [8]. None of them repairs or withholds. Mut4All synthesises mutators from past bug reports [2], and our mutation arm follows its design.

Seed-guided generation, reduction, gating. AFuzz [9] is the origin of our method: given a reference bug, an agent analyses the root cause, hypothesises where else that cause could live, and tests each hypothesis with generated code, finding 40 bugs in V8 in a month. We apply it to a hardware compiler, behind a repair chain and a gate. Delta debugging [10] is the algorithm stage 5’s reducer implements, and we contribute only its placement. Abstain-and-Validate raises repair success by

TABLE II
THE CAMPAIGN’S RESULT, ONE GROUP OF ROWS PER TABLE OF THE
ARTEFACT THE LOOP RENDERS. THE SEEDED-BUG VALIDATION
CONTRIBUTES NOTHING TO THE HEADLINE

Measure	seeded	mutation
<i>Headline</i>		
Distinct confirmed bugs	<i>pending</i>	<i>pending</i>
Confirmed filings	<i>pending</i>	<i>pending</i>
Contaminated candidates, and distinct confirmed without them	<i>pending</i>	<i>pending</i>
<i>The five secondaries</i>		
Candidates before deduplication	<i>pending</i>	<i>pending</i>
Candidates after deduplication	<i>pending</i>	<i>pending</i>
Reports filed	<i>pending</i>	<i>pending</i>
Gate precision, confirmed over filed	<i>pending</i>	<i>pending</i>
Repairs merged	<i>pending</i>	<i>pending</i>
<i>The budget, in its own unit</i>		
Window, seconds	14,400	14,400
Elapsed, seconds	<i>pending</i>	<i>pending</i>
Stopped by	<i>pending</i>	<i>pending</i>
Spend, US dollars	<i>pending</i>	<i>pending</i>
<i>Candidates at the gate, one bucket each</i>		
Unreproducible	<i>pending</i>	<i>pending</i>
Not minimal	<i>pending</i>	<i>pending</i>
Invalid input	<i>pending</i>	<i>pending</i>
Duplicate	<i>pending</i>	<i>pending</i>
Undecided	<i>pending</i>	<i>pending</i>
New bug	<i>pending</i>	<i>pending</i>

Seed	SDK exact	eligible	probes	candidates	filings
<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pending</i>

letting a second model decline a bug and reject a patch [11], llvm-harness packages LLVM’s middle end for agents [12], and CodeRover-S repairs security bugs found by OSS-Fuzz [13]; each makes an existing repair agent more selective, while our gate sits downstream, decides what reaches a human at all, and discards by default.

VII. ARTEFACT AND LIMITATIONS

We release the loop as CHIA nodes and tools, the framework’s own units of composition, so swapping the generator for the mutation arm is one node substituted. Its two halves, the one that proposes work and the one that measures it, meet at a single frozen contract of seven record types and one function call, version 2.0, and are tested separately against recorded examples of it; the suite is 529 tests, of which 490 need no cluster, no network and no model. Four things go upstream to CHIA: the Dockerfile for the assertions-on image, its build workflow, three helpers for CHIA’s own CIRCT module, and the nineteen-line branch that lets its repair chain select the backend this loop runs on. All of it, with the seed corpus and every input, verdict, reduced case, transcript and gate decision, is at https://github.com/DevAbhinav-23/CHIA_Hackathon_2k26.

The apparatus is verified a stage at a time, and the strongest piece of that evidence is the second oracle’s harness generators: from one generated design they emit a CIRCT simulation harness and a Verilator testbench printing identical 112-line sequences, and one changed constant makes them disagree at a named cycle and signal.

What is not built. Nothing has run end to end with a live model: beyond three one-word calls that verified the credential, no model call has been made, the interlock staying closed until the pilot. The 187 seeds rest on a subject-keyword filter, no

diffs read to confirm each is a bug fix; of seven mining attempts, one seed did not fail at its parent. How many seeds the second oracle applies to has not been counted, the contamination screen misses a sibling fixed in a commit whose subject does not name it, and CHIA’s repair chain returns no token counts for its own turns, so the reported spend excludes that stage. The image is built but not published, so a report names its content hash rather than a registry digest.

Scope. The loop does not repair wrong-output bugs, because the oracle that would find them is report-only, and it does not replace maintainers: it converts the human event from writing a bug report into reading one.

ACKNOWLEDGEMENTS

We thank the CHIA developers and the A³ hackathon organisers. *AI-assistance disclosure*. Claude (Anthropic) and Gemini (Google) were used for drafting, code generation and editorial revision; the human authors made every claim, design decision and number their own, and are responsible for the entire contents of this paper.

REFERENCES

- [1] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” *arXiv preprint arXiv:2606.27350*, 2026.
- [2] B. Wang *et al.*, “Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports,” *arXiv preprint arXiv:2507.19275*, 2025.
- [3] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- [4] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011, pp. 283–294.
- [5] C. Suo, J. Chen, S. Liu, J. Jiang, Y. Zhao, and J. Wang, “Fuzzing MLIR compiler infrastructure via operation dependency analysis,” in *Proc. 33rd ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2024, pp. 1287–1299.
- [6] B. Cao, W. Tong, Z. Tang, Z. Wang, H. Huang, and Y. Yan, “Nüwa: Enhancing MLIR fuzzing with LLM-driven generation and adaptive mutation,” in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2025.
- [7] C. Suo, J. Wang, Y. Wang, J. Jiang, Q. Shen, and J. Chen, “DESIL: Detecting silent bugs in MLIR compiler infrastructure,” *arXiv preprint arXiv:2504.01379*, 2025.
- [8] Z. Sun, J. Liang, W. Wang, C. Suo, J. Chen, and F. Xu, “Interleaved learning and exploration: A self-adaptive fuzz testing framework for MLIR,” in *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025, arXiv:2510.07815.
- [9] J. Park and I. Yun, “Agentic fuzzing: Opportunities and challenges,” *arXiv preprint arXiv:2605.10074*, 2026.
- [10] A. Zeller and R. Hildebrandt, “Simplifying and isolating failure-inducing input,” *IEEE Trans. Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [11] J. Cambronerio *et al.*, “Abstain and validate: A dual-LLM policy for reducing noise in agentic program repair,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, arXiv:2510.03217.
- [12] Y. Zheng, C. Li, S. Li, Y. Zhang, and Z. Su, “Agentic harness for real-world compilers,” *arXiv preprint arXiv:2603.20075*, 2026.
- [13] Y. Zhang, J. Wang, D. Berzin, M. Mirchev, and A. Roychoudhury, “Fixing security vulnerabilities with agentic AI in OSS-Fuzz,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, pp. 374–384.